



Astro + Svelte

Islands All The Way Down

A practical guide to building content-rich sites with interactive islands. All examples from a real personal site.

Introduction

This tutorial teaches Astro and Svelte 5 through the lens of a real project: **Absolute Ablation**, a personal site with blog posts, travel guides, and interactive components. You'll learn how Astro's static-first architecture combines with Svelte's reactive islands to create fast, content-rich sites.

Astro is a web framework for content-driven websites. It renders everything to static HTML at build time, with zero JavaScript shipped by default. When you need interactivity, you "opt in" per-component using **islands architecture**.

Svelte 5 is a compiler that turns declarative UI code into surgical DOM updates. No virtual DOM, no runtime diffing. Combined with Astro, Svelte components become interactive islands in a sea of static HTML.

Table of Contents

Introduction	2
Astro Fundamentals	3
Islands Architecture	5
Content Collections	8
MDX + Svelte Components	11
Layouts and Composition	15
Building Interactive Components	18
Putting It All Together	22
Quick Reference Card	24

Audience: Developers who know basic HTML/CSS/JavaScript and want to build content-rich sites with modern tooling. No prior framework experience required.

Approach: Each chapter teaches a concept with real code from the Absolute Ablation site, then explains what happens at build time and runtime.

Astro Fundamentals

Astro is a **static site generator** with a twist: it understands components from multiple frameworks (Svelte, React, Vue) but renders them all to plain HTML at build time. The key insight is that most of a content site is static—text, images, navigation—and only small parts need JavaScript interactivity.

```

graph TD
    A[Static HTML] <--> B[Full SPA]
    A --> C[Astro (default)  
Zero JS shipped]
    C --> D[Add islands]
    D --> E[selective JS]
    B --> F[React/Next.js (default)  
Full framework shipped]
    F --> G[Remove JS]
    G --> H[??]
  
```

Static HTML ← Full SPA

Static HTML path:
 ↓
 Astro (default)
 Zero JS shipped
 ↓
 Add islands → selective JS

Full SPA path:
 ↓
 React/Next.js (default)
 Full framework shipped
 ↓
 Remove JS → ???

The .astro File Format

File: `src/pages/index.astro`

The Code Fence

- Import components and utilities
- Fetch data from APIs, databases, or the filesystem
- Compute values, filter arrays, format dates
- Access environment variables

Template Expressions

Astro + Svelte Tutorial

- `{title}` — inserts a string
- `{posts.map(p => ... )}` — renders a list (JSX-like syntax)
- `{condition && <p>shown</p>}` — conditional rendering
- `{value ?? 'default'}` — nullish coalescing

Unlike React, these expressions run once at build time and produce static HTML. There is no reactivity, no re-rendering, no state management.

File-Based Routing

Every file in `src/pages/` becomes a route:

```
src/pages/
  index.astro      → /
  about.astro      → /about
  blog/
    index.astro    → /blog
    [slug].astro   → /blog/my-post (dynamic)
  travel/
    index.astro    → /travel
    [slug].astro   → /travel/austria-trip
```

- `index.astro` files map to directory roots
- `[slug].astro` files are dynamic routes — the bracket syntax captures a URL parameter
- Nested directories create nested URL paths

Static Output

With output: `'static'` in the config (the default), Astro generates a folder of HTML files:

File: [astro.config.mjs](#)

```
import { defineConfig } from 'astro/config';
import svelte from '@astrojs/svelte';
import mdx from '@astrojs/mdx';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  output: 'static', // default: pure static HTML
  integrations: [svelte(), mdx()],
  vite: {
    plugins: [tailwindcss()],
  },
});
```

The build output is a folder of `.html` files that can be deployed to any static host (Netlify, Vercel, GitHub Pages, S3). No server required.

Gotcha 1: Build-time vs runtime

Code in the fence runs during `astro build`, not when a user visits the page. If you fetch an API in the fence, the data is frozen at build time. For dynamic data, you need client-side fetching (Svelte components).

Gotcha 2: No client-side routing

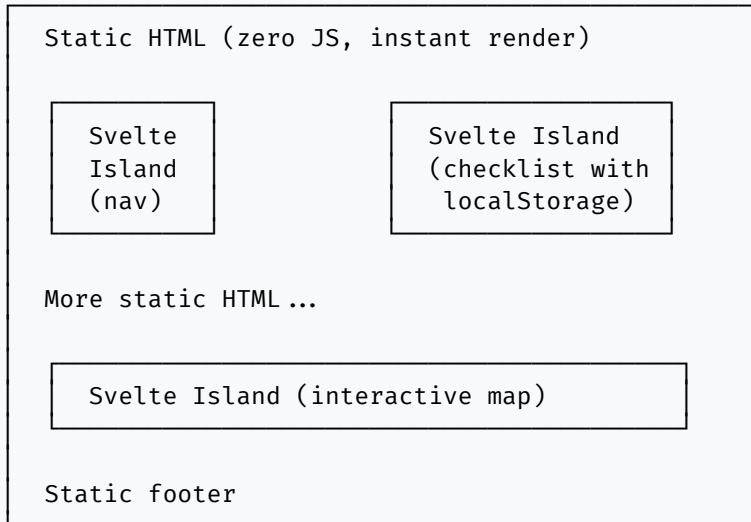
Static Astro sites use full page navigations. Each link loads a new HTML document. This is intentional—it's simpler and faster for content sites. If you need SPA-like transitions, Astro offers View Transitions as an opt-in.

CHAPTER 2

Islands Architecture

The Core Idea

In a traditional SPA, the entire page is one JavaScript application. In Astro's islands architecture, the page is static HTML with isolated **islands** of interactivity. Each island is an independent component that hydrates separately.



Client Directives

You control **when** each island hydrates using `client:` directives:

Directive	When it hydrates	Use for
(no directive)	Never — static HTML only	Content that needs no JS: headings, paragraphs, images, static cards
<code>client:load</code>	Immediately on page load	Above-the-fold interactive elements: navigation, search, critical UI
<code>client:visible</code>	When scrolled into viewport	Below-the-fold: charts, maps, comments, interactive widgets
<code>client:idle</code>	After page is idle	Low-priority: analytics, non-critical enhancements
<code>client:media</code>	When CSS media query matches	Mobile-only or desktop-only interactivity

Example: Static vs Interactive

```
—
import TripPackingChecklist from
  '../components/content/TripPackingChecklist.svelte';
import Callout from
  '../components/content/Callout.svelte';
—

<!-- Static: renders to HTML, ships zero JS -->
<Callout type="tip">
```

```
Pack light – you can buy supplies at DM or Rossmann.  
</Callout>
```

```
<!-- Interactive: ships Svelte JS, hydrates on load -->  
<TripPackingChecklist  
  client:load  
  storageKey="austria-packing"  
>
```

The Callout component has no `client:load` directive. Astro renders it to static HTML at build time and throws away the component code. The browser receives a `<div>` with styled text. Zero JavaScript.

The TripPackingChecklist has `client:load`. Astro renders the initial HTML at build time (for fast first paint), then ships the Svelte component JS. On page load, Svelte “hydrates” the static HTML — attaching event listeners, restoring `localStorage` state, and making checkboxes interactive.

The Performance Win

A typical content page might have 20 components but only 2–3 need interactivity. Without islands, you’d ship JavaScript for all 20. With islands, you ship JS only for the 2–3 that need it:

Traditional SPA:

Page JS = Framework runtime + ALL component code
~150–300 KB (gzipped) typical

Astro with islands:

Page JS = Svelte runtime (~5 KB) + interactive components only
~10–30 KB (gzipped) typical

The difference is not optimization. It’s architecture.

How Hydration Works

When Astro builds a page with a `client:load` component:

1. **Build time:** Astro calls the Svelte compiler, renders the component to HTML, and embeds it in the page
2. **Build time:** Astro generates a small script tag that imports the component JS
3. **Runtime:** Browser loads the static HTML (fast first paint)
4. **Runtime:** The script loads the Svelte component code
5. **Runtime:** Svelte “hydrates” — walks the existing DOM, attaches event listeners, connects reactive state

The user sees content immediately (step 3) and gets interactivity moments later (step 5). This is fundamentally different from an SPA where the user sees a blank page until JavaScript loads and renders everything.

Islands are independent

Each island hydrates independently. A slow-loading island doesn’t block other islands or the static content. If an island’s JS fails to load, the rest of the page still works perfectly.

Gotcha 1: Forgetting `client:load` directive

If you use a Svelte component without `client:load` (or similar), it renders to static HTML. Event handlers, `$state`, `$effect` — none of it works. The component is “dead” HTML. This is the 1 beginner mistake.

Gotcha 2: Choosing the wrong directive

`client:load` is the safe default. Use `client:visible` for below-the-fold content to reduce initial JS. Don't over-optimize — `client:idle` can cause visible delays for components the user interacts with immediately.

Content Collections

The Problem

A blog or travel site has many content files (Markdown, MDX) with structured frontmatter. You need to:

1. Validate that every post has the required fields (title, date, tags)
2. Query posts (filter by tag, sort by date)
3. Generate pages from content files

Content Collections solve all three with type-safe schemas.

Defining a Schema

File: [src/content.config.ts](#)

```
import { defineCollection, z } from 'astro:content';
import { glob } from 'astro/loaders';

const blog = defineCollection({
  loader: glob({ pattern: '**/*.md,mdx', base: './src/content/blog' }),
  schema: z.object({
    title: z.string(),
    description: z.string(),
    pubDate: z.coerce.date(),
    updatedAt: z.coerce.date().optional(),
    heroImage: z.string().optional(),
    tags: z.array(z.string()).default([]),
    draft: z.boolean().default(false),
  }),
});

const travel = defineCollection({
  loader: glob({ pattern: '**/*.md,mdx', base: './src/content/travel' }),
  schema: z.object({
    title: z.string(),
    description: z.string(),
    pubDate: z.coerce.date(),
    updatedAt: z.coerce.date().optional(),
    heroImage: z.string().optional(),
    tripDate: z.string().optional(),
    tags: z.array(z.string()).default([]),
    draft: z.boolean().default(false),
  }),
});

export const collections = { blog, travel };
```

Line by Line

- `defineCollection({ ... })` — Registers a collection with its loader and schema.
- `loader: glob({ ... })` — Tells Astro where to find content files. The glob pattern matches Markdown and MDX files.
- `schema: z.object({ ... })` — A **Zod** schema that validates frontmatter. If a post is missing a required field, the build fails with a clear error.
- `z.coerce.date()` — Coerces strings like "2026-05-23" into JavaScript Date objects. Critical because YAML frontmatter parses dates as strings.

- `z.array(z.string()).default([])` — An array of strings, defaulting to empty if omitted.
- `z.boolean().default(false)` — A boolean with a default value.

Querying Collections

In any `.astro` page's code fence:

```

—
import { getCollection } from 'astro:content';

// Get all non-draft travel posts, sorted by date
const posts = (await getCollection('travel'))
  .filter(post => !post.data.draft)
  .sort((a, b) =>
    b.data.pubDate.valueOf() - a.data.pubDate.valueOf()
  );
—

{posts.map(post => (
  <a href={`~/travel/${post.slug}`}>
    <h2>{post.data.title}</h2>
    <time>{post.data.pubDate.toLocaleDateString()}</time>
  </a>
))}

```

- `getCollection('travel')` — Returns all entries in the travel collection, with validated and typed data (frontmatter) and body (content).
- `post.data.title` — Fully typed. TypeScript knows title is a string because the schema says so. Accessing `post.data.bogus` is a compile error.
- `post.slug` — Auto-generated from the filename (e.g., `rotterdam-to-austria.mdx` becomes `rotterdam-to-austria`).

Rendering a Collection Entry

File: `src/pages/travel/[slug].astro`

```

—
import { getCollection, render } from 'astro:content';
import BlogLayout from '../layouts/BlogLayout.astro';

export async function getStaticPaths() {
  const posts = await getCollection('travel');
  return posts.map(post => ({
    params: { slug: post.id },
    props: { post },
  }));
}

const { post } = Astro.props;
const { Content } = await render(post);
—

<BlogLayout title={post.data.title}
  description={post.data.description}>
  <Content />
</BlogLayout>

```

How `getStaticPaths` Works

For dynamic routes ([`slug`].astro), Astro needs to know **all possible values** at build time. `getStaticPaths()` returns an array of `{ params, props }` objects:

- `params: { slug: post.id }` — Tells Astro to generate `/travel/rotterdam-to-austria`
- `props: { post }` — Passes the full post object to the page

Astro calls `getStaticPaths()` once during build, generates one HTML file per entry, and never calls it again. This is fundamentally different from Next.js's `getServerSideProps` which runs on every request.

The `render()` Function

`await render(post)` compiles the Markdown/MDX body into an Astro component. The returned Content component renders the post body including any imported Svelte components (in MDX files).

Gotcha 1: `z.coerce.date()` is essential

YAML frontmatter like `pubDate: 2026-05-23` parses as a string, not a Date. Without `z.coerce.date()`, comparisons and formatting break silently.

Gotcha 2: Collection names must match exports

The key in `export const collections = { blog, travel }` must match the directory name under `src/content/`. Mismatches cause silent failures.

Gotcha 3: Draft filtering is manual

Astro doesn't filter drafts automatically. You must add `.filter(p => !p.data.draft)` in your queries.

MDX + Svelte Components

What is MDX?

MDX is Markdown that can import and render components. Regular Markdown gives you headings, paragraphs, links, code blocks. MDX adds the ability to embed interactive Svelte components directly in your content.

```
Regular Markdown:
# My Post
Some text with bold and [links]().

MDX:
# My Post
Some text with bold and [links]().

import Chart from '../components/Chart.svelte';
<Chart data={[1,2,3]} client:visible />
```

Importing Components in MDX

File: [src/content/travel/rotterdam-to-austria.mdx](#)

```
—
title: "Rotterdam to Austria"
description: "11-day family road trip"
pubDate: 2026-05-23
tags: ["travel", "austria", "road-trip"]
—

import TripPackingChecklist from
  '../..components/content/TripPackingChecklist.svelte';
import TravelTips from
  '../..components/content/TravelTips.svelte';
import Callout from
  '../..components/content/Callout.svelte';
import DataTable from
  '../..components/content/DataTable.svelte';

# Rotterdam to St. Wolfgang

An 11-day family road trip through Germany and Austria...
```

Passing Props: Strings and Numbers

```
<Callout type="warning">
  Whit Monday (May 25) is a public holiday.
  All shops closed in Germany AND Austria.
</Callout>
```

String props use quotes. The text between opening and closing tags becomes the component's children.

Passing Props: Arrays and Objects

For complex data, define JavaScript variables and pass them as props:

```
export const destinations = [
  "Netherlands",
  "Germany",
  "Austria"
];

<TravelTips
  client:load
  storageKey="austria-tips"
  destinations={destinations}
/>
```

Passing Props: Nested Objects

```
export const packingExtras = {
  "Austria Legal Requirements": [
    "First aid kit (mandatory in Austria)",
    "Warning triangle",
    "High-visibility vest",
  ],
  "Road Trip Essentials": [
    "Austrian motorway vignette",
    "Green card (insurance)",
  ],
};

<TripPackingChecklist
  client:load
  storageKey="austria-packing"
  extraItems={packingExtras}
/>
```

The `export const` syntax creates a JavaScript variable in the MDX file's scope. This variable is then available as a prop value using curly braces.

Passing Props: Complex Configuration

```
export const timelineColumns = [
  { key: "when", label: "When", width: "25%" },
  { key: "task", label: "Task", width: "45%" },
  { key: "notes", label: "Notes", width: "30%" },
];

export const timelineData = [
  { when: "6 months before", task: "Book accommodation",
    notes: "St. Wolfgang fills up fast" },
  { when: "2 months before", task: "Buy vignette online",
    notes: "asfinag.at – 10-day vignette" },
  { when: "1 week before", task: "Pack car supplies",
    notes: "First aid kit, triangle, vest" },
];

<DataTable
  client:visible
  columns={timelineColumns}
  data={timelineData}
```

```
caption="Pre-Trip Timeline"  
>
```

The Children / Snippet Pattern

Svelte 5 uses `{@render children()}` to render content passed between component tags. In MDX, this means Markdown content between tags:

```
<Callout type="tip">  
  This Markdown content becomes the `children` snippet.  
  You can use bold, italic, and [links]().  
</Callout>
```

The Svelte component receives this as a children snippet:

```
<script lang="ts">  
  import type { Snippet } from 'svelte';  
  interface Props {  
    type?: 'tip' | 'warning' | 'insight';  
    children?: Snippet;  
  }  
  let { type = 'tip', children }: Props = $props();  
</script>  
  
<div class="callout callout-{type}">  
  {#if children}  
    <div class="prose-content">  
      {@render children()}  
    </div>  
  {/if}  
</div>
```

The prose-content Pattern

When Markdown is rendered inside a Svelte component, it arrives as raw HTML (processed by the MDX compiler). The prose-content wrapper div lets you apply Tailwind Typography styles:

```
/* In your global CSS */  
.prose-content {  
  @apply prose prose-sm max-w-none;  
}
```

Without this, the Markdown HTML inside components loses all styling (headings look like plain text, lists have no bullets).

Gotcha 1: No Markdown inside component tags (directly)

MDX has a critical limitation: Markdown syntax inside component tags is only processed if there's a blank line after the opening tag and before the closing tag:

```
❌— BROKEN: Markdown not processed →  
<Callout type="tip">  
**Bold text** and [links]()  
</Callout>  
  
✅— WORKS: blank line triggers Markdown processing →  
<Callout type="tip">  
  
**Bold text** and [links]()  
  
</Callout>
```

The blank lines are required for the MDX compiler to switch from "JSX mode" to "Markdown mode" within component tags.

Gotcha 2: Import paths are relative to the MDX file

Imports in MDX use paths relative to the .mdx file's location, not the project root. From `src/content/travel/my-post.mdx`, a component at `src/components/Foo.svelte` is imported as `../.. /components/Foo.svelte`.

Gotcha 3: export const for complex props

You cannot write inline object literals in JSX-like syntax within MDX. Always use `export const` for arrays and objects, then reference them by name.

Layouts and Composition

The Layout Stack

Astro uses a **composition** pattern for layouts: each layout wraps its content in a `<slot />`, and layouts can nest inside each other. A typical blog site has two layers:

```
Layout.astro (base: html, head, body, global styles)
├─ BlogLayout.astro (blog-specific: sidebar, TOC, article)
└─ <Content /> (the actual MDX content)
```

The Base Layout

File: `src/layouts/Layout.astro`

```
—
interface Props {
  title: string;
  description?: string;
}

const { title, description } = Astro.props;
—

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport"
      content="width=device-width, initial-scale=1" />
    <title>{title}</title>
    {description &&
      <meta name="description" content={description} />}
  </head>
  <body>
    <slot />
  </body>
</html>
```

How slot Works

The `<slot />` element is a placeholder. Whatever the parent passes between the component's opening and closing tags replaces the slot. It's the same concept as Svelte's `{@render children()}`, but in Astro's `.astro` files it uses the HTML-native `<slot>` syntax.

The Blog Layout

File: `src/layouts/BlogLayout.astro`

```
import Layout from './Layout.astro';

interface Props {
  title: string;
  description?: string;
  pubDate?: Date;
}

const { title, description, pubDate } = Astro.props;

<Layout title={title} description={description}>
  <main class="max-w-4xl mx-auto px-4">
    <article>
      <header>
        <h1>{title}</h1>
        {pubDate && (
          <time datetime={pubDate.toISOString()}>
            {pubDate.toLocaleDateString('en-US', {
              year: 'numeric',
              month: 'long',
              day: 'numeric'
            })}
          </time>
        )}
      </header>
      <div class="prose">
        <slot />
      </div>
    </article>
  </main>
</Layout>
```

The Rendering Chain

When Astro builds a travel post, the full chain is:

1. `getStaticPaths()` in `[slug].astro` yields all travel posts
2. For each post, `[slug].astro` renders `<BlogLayout><Content /></BlogLayout>`
3. `BlogLayout.astro` wraps the content in article markup and passes it to `Layout.astro`
4. `Layout.astro` wraps everything in `<html><head> ... <body> ... </body></html>`
5. The MDX `<Content />` component renders Markdown to HTML and hydrates Svelte islands

The result is a single `.html` file with:

- Proper `<head>` metadata
- Blog-specific layout (header, article wrapper)
- Rendered Markdown content
- Script tags for interactive Svelte islands

Astro.props

Every `.astro` component receives props via `Astro.props`. This is the build-time equivalent of Svelte's `$props()`:


```

—
// Defining props with TypeScript
interface Props {
  title: string;
  count?: number;
}
const { title, count = 0 } = Astro.props;
—

<h1>{title} ({count})</h1>

```

The interface `Props` is a convention that enables TypeScript autocompletion when using the component.

Named Slots

Astro supports named slots for multi-section layouts:

```

<!-- Layout with named slots -->
<header>
  <slot name="hero" />
</header>
<main>
  <slot /> <!-- default slot -->
</main>
<aside>
  <slot name="sidebar" />
</aside>

<!-- Using named slots -->
<MyLayout>
  <div slot="hero">Hero content</div>
  <div slot="sidebar">Sidebar content</div>
  <p>This goes in the default slot</p>
</MyLayout>

```

Gotcha 1: Astro slots vs Svelte snippets

Astro `.astro` files use `<slot />` (HTML standard). Svelte `.svelte` files use `{#snippet} + {@render}` (Svelte 5 syntax). Don't mix them — use the right syntax for the right file type.

Gotcha 2: Props flow one direction

Like Svelte, Astro props flow from parent to child. A layout cannot reach up into the page that uses it. Data flows down via props, content flows down via slots.

Building Interactive Components

Design Principle: Generic Components

The most important design decision for reusable components: **never hardcode trip-specific or content-specific data inside the component**. All data should come from props, and the MDX file should pass it.

This means the same `TripPackingChecklist` component works for Switzerland, Austria, or any future trip — the component handles the interactive behavior (checkboxes, `localStorage` persistence, progress tracking), while the MDX file provides the trip-specific items.

Anatomy of an Interactive Component

File: `src/components/content/TripPackingChecklist.svelte`

```
<script lang="ts">
  interface CategoryDef {
    name: string;
    items: string[];
  }

  interface Props {
    storageKey?: string;
    extraItems?: Record<string, string[]>;
    extraCategories?: CategoryDef[];
  }

  let {
    storageKey = 'trip-packing',
    extraItems = {},
    extraCategories = [],
  }: Props = $props();

  // Base packing items (shared across all trips)
  const baseCategories: CategoryDef[] = [
    {
      name: "Documents",
      items: [
        "Passports",
        "Driver's license",
        "Insurance cards",
        "Emergency contact card",
        "Green card (vehicle insurance)",
      ],
    },
    // ... more base categories
  ];

  // Merge base + extras
  let allCategories = $derived.by(() => {
    let cats = [...baseCategories];
    // Merge extraItems into existing categories
    for (const [catName, items] of
      Object.entries(extraItems)) {
      const existing = cats.find(
        c => c.name === catName);
      if (existing) {
        existing.items = [
          ...existing.items, ...items];
      } else {
        cats.push({ name: catName, items });
      }
    }
    // Append extra categories
    return [...cats, ...extraCategories];
  });
</script>
```

Key Patterns

- **Props with defaults:** `storageKey = 'trip-packing'` — sensible default, overridable per trip
- **Generic data model:** `extraItems` and `extraCategories` let each MDX file inject trip-specific content
- **`$derived.by()` for merging:** The merged list recomputes whenever props change (though in practice, props are static in MDX)
- **localStorage persistence:** The component saves checked state using the `storageKey`, so users don't lose progress on page reload

Using the Component in MDX

```
export const austriaExtras = {
  "Documents": [
    "Austrian motorway vignette receipt",
  ],
  "Austria Legal Requirements": [
    "First aid kit (mandatory in Austria)",
    "Warning triangle (mandatory in Austria)",
    "High-visibility vest (mandatory in Austria)",
  ],
};

<TripPackingChecklist
  client:load
  storageKey="austria-2026-packing"
  extraItems={austriaExtras}
/>
```

The same component in a different trip:

```
export const swissExtras = {
  "Documents": [
    "Swiss motorway vignette receipt",
  ],
};

<TripPackingChecklist
  client:load
  storageKey="switzerland-2026-packing"
  extraItems={swissExtras}
/>
```

Different `storageKey` = different `localStorage` entry = independent check state per trip. Different `extraItems` = different trip-specific packing items. Same component.

The `storageKey` Pattern

Every interactive component that persists state should accept a `storageKey` prop:

```
<script lang="ts">
  let { storageKey = 'default' }: Props = $props();

  // Load from localStorage
  let checked: Set<string> = $state(new Set());

  $effect(() => {
```

```

    const saved = localStorage.getItem(storageKey);
    if (saved) {
      checked = new Set(JSON.parse(saved));
    }
  });

  // Save to localStorage when checked changes
  $effect(() => {
    localStorage.setItem(
      storageKey,
      JSON.stringify([ ...checked ])
    );
  });
</script>

```

This pattern ensures:

- Each instance of the component has isolated state
- State survives page reloads
- Multiple instances on the same page don't conflict

Gotcha 1: `client:load` is required for `localStorage`

`localStorage` is a browser API. Without `client:load`, the component is static HTML and has no access to browser APIs. The checkboxes render but don't persist.

Gotcha 2: Unique `storageKey` per instance

If two components share a `storageKey`, they share state. Always use unique keys: `"austria-packing"`, `"switzerland-packing"`, never just `"packing"`.

Putting It All Together

The Full Architecture

```

src/
  content.config.ts           Schema definitions (Zod)
  content/
    travel/
      rotterdam-to-austria.mdx MDX content + Svelte islands
      rotterdam-to-switzerland.mdx
    blog/
      ...
  components/
    content/
      TripPackingChecklist.svelte  Generic, prop-driven
      TravelTips.svelte            Generic, prop-driven
      Callout.svelte               Static (no client:)
      DataTable.svelte             Interactive tables
  layouts/
    Layout.astro                 Base HTML shell
    BlogLayout.astro             Blog article wrapper
  pages/
    travel/
      index.astro                 List all travel posts
      [slug].astro                Render individual post

```

The Build Pipeline

When you run `astro build`:

1. **Schema validation:** Astro validates every content file's frontmatter against the Zod schema. Invalid files fail the build.
2. **Content compilation:** MDX files are compiled — Markdown becomes HTML, `import` statements are resolved, `export const` variables are evaluated.
3. **Static path generation:** `getStaticPaths()` runs for every `[slug].astro` file, generating the full list of pages.
4. **Page rendering:** Each page renders its layout chain (Layout → BlogLayout → Content), producing complete HTML.
5. **Island extraction:** Components with `client:` directives get their JavaScript bundled separately. Astro generates script tags that lazy-load these bundles.
6. **Output:** A `dist/` folder of HTML files, CSS, and minimal JavaScript.

What Ships to the Browser

For a typical travel post:

Asset	Size (approx)	Purpose
HTML document	15–30 KB	Full page content, pre-rendered
CSS (Tailwind)	8–15 KB gzipped	Styling
Svelte runtime	5 KB gzipped	Hydration + reactivity

Asset	Size (approx)	Purpose
Interactive components	3–8 KB gzipped	Checklist, tips, tables
Total	31–58 KB	Complete interactive page

Compare this to a React SPA where the framework alone is 44 KB gzipped, before any application code.

Key Takeaways

- **Astro renders everything to static HTML by default.** You add JavaScript only where needed.
- **Content Collections give you typed, validated content.** Frontmatter errors fail the build, not the user's browser.
- **MDX bridges content and components.** Write Markdown for prose, embed Svelte for interactivity.
- **Islands architecture is not an optimization — it's the default.** You don't "lazy load" components; you "opt in" to JavaScript.
- **Generic components with props** work across all content. The MDX file owns the data; the component owns the behavior.
- **Layouts compose via slots.** Each layer handles its responsibility: base HTML, page structure, content rendering.

The Decision Framework

If you need...	Use...	Example
Static content display	No client: directive	Callout, SectionHeading
User interaction	client:load	PackingChecklist, TravelTips
Below-fold interactive	client:visible	DataTable, charts
Content with frontmatter	Content Collection + MDX	Blog posts, travel guides
Shared page structure	Astro Layout	Header, footer, nav
Reusable template blocks	Svelte Snippets	Card templates, list items
Trip-specific data	MDX export const + props	Packing extras, timelines

Quick Reference Card

Astro Essentials

Concept	Syntax	Notes
Code fence	<code>— ... —</code>	Build-time JS. Runs once during build.
Template expression	<code>{expression}</code>	Evaluated at build time, produces static HTML
Component import	<code>import X from '...'</code>	In code fence. Works with <code>.astro</code> , <code>.svelte</code> , <code>.tsx</code>
Props	<code>Astro.props</code>	Received from parent component
Slot	<code><slot /></code>	Placeholder for child content
Named slot	<code><slot name="x" /></code>	Multiple content insertion points
Static paths	<code>getStaticPaths()</code>	Generates all pages for dynamic routes
Collection query	<code>getCollection('name')</code>	Returns all entries, typed by schema
Render content	<code>await render(post)</code>	Compiles MDX/MD to renderable component

Client Directives

Directive	Hydrates	Use for
(none)	Never	Static display components
<code>client:load</code>	Page load	Critical interactive elements
<code>client:visible</code>	Enters viewport	Below-fold interactive content
<code>client:idle</code>	Browser idle	Low-priority enhancements
<code>client:media</code>	Media query match	Responsive interactivity

MDX Patterns

Pattern	Example
Import component	<code>import Foo from '../..components/Foo.svelte';</code>
String prop	<code><Foo type="warning" /></code>
Expression prop	<code><Foo count={42} data={myArray} /></code>
Complex data	<code>export const data = [...]; then <Foo data={data} /></code>
Children content	<code><Foo></code> Markdown between tags (needs blank lines) <code></Foo></code>
Interactive island	<code><Foo client:load prop={value} /></code>

All code examples from the Absolute Ablation personal site.